

```
//@ HAMR: --platform Microkit --sel4-output-dir ../hamr/microkit
```

```
//
```

```
=====
=====
```

```
// Project: Micro-ROS Seed Imaging System
```

```
// File:  MicroROS_Schematic.sysml
```

```
// Notes: SysML v2 (HAMR / AADL flavour) representation of the wiring
```

```
//      schematic. Every thread carries a GUMBO contract block that
```

```
//      encodes its verification details (assume / guarantee), in line
```

```
//      with the BASS isolate-ethernet-simple SW.sysml convention.
```

```
//
```

```
=====
=====
```

```
package MicroROS {
```

```
    private import HAMR::*;
```

```
    //
```

```
=====
=
```

```
    // Data Types
```

```
    //
```

```
=====
=
```

```
    enum def AxisRole {
```

```
enum Z_TURNTABLE;  
enum X_TILT;  
enum Y_RESERVED;  
}
```

```
enum def MotorDirection {  
    enum CW;  
    enum CCW;  
    enum STOP;  
}
```

```
enum def CoilLine {  
    enum IN1;  
    enum IN2;  
    enum IN3;  
    enum IN4;  
}
```

```
enum def PowerRailState {  
    enum OFF;  
    enum NOMINAL_12V;  
    enum BROWNOUT;  
    enum OVERVOLTAGE;  
}
```

```
enum def ModeType {
```

```
enum Init;  
enum Idle;  
enum Imaging;  
enum Fault;  
}
```

```
enum def ErrorStatusType {  
    enum Ok;  
    enum Failed;  
}
```

```
part def StepCommand_i :> Data {  
    attribute axis    : AxisRole;  
    attribute direction : MotorDirection;  
    part    stepCount : Base_Types::Unsigned_32;
```

```
language "GUMBO" /*{  
    invariants  
        inv stepCountsBounded:  
            stepCount <= 200000;  
            // a single command never asks for more than 200k steps  
    */  
}
```

```
part def CoilDriveFrame_i :> Data {  
    attribute axis : AxisRole;
```

```
attribute in1 : Base_Types::Boolean;
```

```
attribute in2 : Base_Types::Boolean;
```

```
attribute in3 : Base_Types::Boolean;
```

```
attribute in4 : Base_Types::Boolean;
```

```
language "GUMBO" /*{
```

```
  invariants
```

```
    inv noShootThroughCoilA:
```

```
      not (in1 == true & in2 == true);
```

```
    inv noShootThroughCoilB:
```

```
      not (in3 == true & in4 == true);
```

```
    // both halves of an H-bridge must never be high simultaneously
```

```
*/
```

```
}
```

```
part def CameraFrame_i :> Data {
```

```
  attribute widthPx  : Base_Types::Unsigned_32;
```

```
  attribute heightPx  : Base_Types::Unsigned_32;
```

```
  attribute timestampUs : Base_Types::Unsigned_64;
```

```
  part  pixelData  : RawPixelBuffer;
```

```
language "GUMBO" /*{
```

```
  invariants
```

```
    inv frameIsHD: widthPx == 1920 & heightPx == 1080;
```

```
*/
```

```
}
```

```

part def RawPixelBuffer :> Array {
  part :>> Base_Type : Base_Types::Unsigned_8;
  attribute :>> Dimensions = 6220800;    // 1920 x 1080 x 3
  attribute :>> Array_Size_Kind = Array_Size_Kinds::Fixed;
  attribute :>> Data_Size = 6220800[byte];
}

```

```

part def PowerSample_i :> Data {
  attribute railVoltage_mV : Base_Types::Unsigned_32;
  attribute state      : PowerRailState;
}

```

```

language "GUMBO" /*{
  invariants
    inv stateMatchesVoltage:
      (state == PowerRailState.NOMINAL_12V implies
        (railVoltage_mV >= 11400 & railVoltage_mV <= 12600)) &
      (state == PowerRailState.BROWNOUT  implies railVoltage_mV < 11400) &
      (state == PowerRailState.OVERVOLTAGE implies railVoltage_mV > 12600);
}*/
}

```

```

//
=====
=

// Threads (verification via GUMBO)

```

```

//
=====

=

part def Periodic_Rust_Thread :> Thread {
    attribute :>> Period          = 1000[ms];
    attribute :>> Dispatch_Protocol = Supported_Dispatch_Protocols::Periodic;
    attribute :>> Implementation_Language = Implementation_Languages::Rust;
    attribute :>> Compute_Execution_Time = 100[ms] .. 100[ms] as
Compute_Execution_Time;
    attribute :>> Stack_Size      = 1048576[byte] as Size;
}

// -----
// ModeManager — system state machine (Init / Idle / Imaging / Fault)
// -----

part def ModeManager_i :> Periodic_Rust_Thread {
    port error_status: DataPort { in :>> type : ErrorStatusType; }
    port mode:      DataPort { out :>> type : ModeType; }

    language "GUMBO" /*{
        state
            current_mode: MicroROS::ModeType;

        initialize
            guarantee modeStartsInInit:
                current_mode == MicroROS::ModeType.Init;

```

compute

guarantee modeOnOutput "Current mode is published every activation":

mode == current_mode;

guarantee initToIdle "Init + Ok -> Idle":

'->:'(In(current_mode) == MicroROS::ModeType.Init &
error_status == MicroROS::ErrorStatusType.Ok,
current_mode == MicroROS::ModeType.Idle);

guarantee initToFault "Init + Failed -> Fault":

'->:'(In(current_mode) == MicroROS::ModeType.Init &
error_status == MicroROS::ErrorStatusType.Failed,
current_mode == MicroROS::ModeType.Fault);

guarantee idleToImaging "Idle + Ok -> Imaging":

'->:'(In(current_mode) == MicroROS::ModeType.Idle &
error_status == MicroROS::ErrorStatusType.Ok,
current_mode == MicroROS::ModeType.Imaging);

guarantee anyToFault "Any mode + Failed -> Fault":

'->:'(error_status == MicroROS::ErrorStatusType.Failed,
current_mode == MicroROS::ModeType.Fault);

guarantee faultIsSticky "Fault never auto-clears":

'->:'(In(current_mode) == MicroROS::ModeType.Fault,

```

        current_mode == MicroROS::ModeType.Fault);

    }*/
}

// -----
// Power Monitor — watches the 12 V rail of the daisy-chained L298N bus
// -----

part def PowerMonitor_i :> Periodic_Rust_Thread {
    port powerSampleIn: EventDataPort { in :>> type : PowerSample_i; }
    port error_status: DataPort { out :>> type : ErrorStatusType; }

    language "GUMBO" /*{
        compute

        assume sampleArrives "Power monitor receives a sample each tick":
            HasEvent(powerSampleIn);

        guarantee MR_HLR_1 "1.1 power: nominal 12 V rail reports Ok":
            (HasEvent(powerSampleIn) implies
                '->:(powerSampleIn.state == MicroROS::PowerRailState.NOMINAL_12V,
                    error_status == MicroROS::ErrorStatusType.Ok));

        guarantee MR_HLR_2 "1.2 power: brownout / overvoltage / off -> Failed":
            (HasEvent(powerSampleIn) implies
                '->:(powerSampleIn.state == MicroROS::PowerRailState.BROWNOUT |
                    powerSampleIn.state == MicroROS::PowerRailState.OVERVOLTAGE |
                    powerSampleIn.state == MicroROS::PowerRailState.OFF,

```



```

error_status == MicroROS::ErrorStatusType.Failed));
}*/
}

// -----
// Camera Driver — 1080p UVC frame producer (USB-C to Jetson)
// -----

part def CameraDriver_i :> Periodic_Rust_Thread {
  port cameraFrameOut: EventDataPort { out :>> type : CameraFrame_i; }
  attribute :>> Period = 33[ms];          // ~30 fps
}

// GUMBO contract is held in a sister specification thread so we keep the
// driver's port surface minimal — same idiom BASS uses for hardware-near
// drivers like LowLevelEthernetDriver_i.

part def CameraDriverContract_i :> Periodic_Rust_Thread {
  port cameraFrameOut: EventDataPort { out :>> type : CameraFrame_i; }

  language "GUMBO" /*{
    compute
      guarantee MR_HLR_3 "1.3 camera: every dispatch publishes one frame":
        MustSend(cameraFrameOut);

      guarantee MR_HLR_4 "1.4 camera: published frames are 1920x1080":
        (MustSend(cameraFrameOut) implies
          cameraFrameOut.widthPx == 1920 &

```

```

        cameraFrameOut.heightPx == 1080);

    }*/
}

// -----
//  micro-ROS Agent — bridges Jetson (ROS 2) to ESP32 (DDS-XRCE)
// -----

part def MicroRosAgent_i :> Periodic_Rust_Thread {
    port stepCommandIn:  EventDataPort { in :>> type : StepCommand_i; }
    port stepCommandOut: EventDataPort { out :>> type : StepCommand_i; }
    port cameraFrameIn:  EventDataPort { in :>> type : CameraFrame_i; }
    port mode_in:        DataPort      { in :>> type : ModeType; }

    language "GUMBO" /*{
        compute

        assume oneCommandPerTick "At most one command per dispatch":
            (not HasEvent(stepCommandIn)) | (HasEvent(stepCommandIn));

        guarantee MR_HLR_5 "1.5 agent: forward step commands only when Imaging":
            (HasEvent(stepCommandIn) implies
                '->:(mode_in == MicroROS::ModeType.Imaging,
                    MustSend(stepCommandOut) and
                    stepCommandOut == stepCommandIn));

        guarantee MR_HLR_6 "1.6 agent: drop step commands outside Imaging":
            (HasEvent(stepCommandIn) implies

```

```

        '->:(mode_in != MicroROS::ModeType.Imaging,
            NoSend(stepCommandOut)));

    }*/
}

// -----
// Stepper Coordinator — runs on ESP32, schedules per-axis coil drives
// -----

part def StepperCoordinator_i :> Periodic_Rust_Thread {

    port stepCommandIn:  EventDataPort { in :>> type : StepCommand_i; }
    port driveZ_out:     EventDataPort { out :>> type : CoilDriveFrame_i; }
    port driveX_out:     EventDataPort { out :>> type : CoilDriveFrame_i; }
    port driveY_out:     EventDataPort { out :>> type : CoilDriveFrame_i; }

    language "GUMBO" /*{

        compute

        assume oneCommandPerTick "At most one step command per dispatch":
            (not HasEvent(stepCommandIn)) | (HasEvent(stepCommandIn));

        guarantee MR_HLR_7 "1.7 coordinator: route Z command to driveZ_out only":
            (HasEvent(stepCommandIn) implies
                '->:(stepCommandIn.axis == MicroROS::AxisRole.Z_TURNABLE,
                    MustSend(driveZ_out) and
                    NoSend(driveX_out) and NoSend(driveY_out)));

        guarantee MR_HLR_8 "1.8 coordinator: route X command to driveX_out only":

```

(HasEvent(stepCommandIn) implies

```
'->: '(stepCommandIn.axis == MicroROS::AxisRole.X_TILT,  
      MustSend(driveX_out) and  
      NoSend(driveZ_out) and NoSend(driveY_out));
```

guarantee MR_HLR_9 "1.9 coordinator: route Y command to driveY_out only":

(HasEvent(stepCommandIn) implies

```
'->: '(stepCommandIn.axis == MicroROS::AxisRole.Y_RESERVED,  
      MustSend(driveY_out) and  
      NoSend(driveZ_out) and NoSend(driveX_out));
```

```
*/
```

```
}
```

```
// -----
```

```
// L298N Driver — one instance per axis, drives D2..D13 GPIO -> IN1..IN4
```

```
// -----
```

```
part def L298N_Driver_i :> Periodic_Rust_Thread {
```

```
  port coilDriveIn: EventDataPort { in :>> type : CoilDriveFrame_i; }
```

```
language "GUMBO" /*{
```

```
  functions
```

```
  def shootThroughA(v: CoilDriveFrame_i): Base_Types::Boolean :=
```

```
    v.in1 == true & v.in2 == true;
```

```
  def shootThroughB(v: CoilDriveFrame_i): Base_Types::Boolean :=
```

```
    v.in3 == true & v.in4 == true;
```

```
  def isSafeFrame(v: CoilDriveFrame_i): Base_Types::Boolean :=
```

(not shootThroughA(v)) & (not shootThroughB(v));

compute

assume oneFramePerTick "At most one drive frame per dispatch":

(not HasEvent(coilDriveIn)) | (HasEvent(coilDriveIn));

guarantee MR_HLR_10 "1.10 driver: only safe (no shoot-through) frames are accepted":

(HasEvent(coilDriveIn) implies isSafeFrame(coilDriveIn));

guarantee MR_HLR_11 "1.11 driver: ENA / ENB jumpers are required for output":

(HasEvent(coilDriveIn) implies

(coilDriveIn.in1 == true | coilDriveIn.in2 == true |

coilDriveIn.in3 == true | coilDriveIn.in4 == true));

// i.e. at least one IN line is asserted when the driver fires

*/

}

// -----

// NEMA-17 Stepper — verification of the physical motor (coil pairing,

// bounded current). Modeled as a periodic thread the same way BASS

// models the LowLevelEthernetDriver_i.

// -----

part def NEMA17_Stepper_i :> Periodic_Rust_Thread {

port coilDriveIn: EventDataPort { in :>> type : CoilDriveFrame_i; }

attribute coilResistance_mOhm : Base_Types::Unsigned_32 = 3000;

```
attribute ratedCurrent_mA : Base_Types::Unsigned_32 = 1500;
```

```
language "GUMBO" /*{
```

```
  compute
```

```
    assume frameIsSafe "Driver only forwards safe frames (no shoot-through)":
```

```
      (HasEvent(coilDriveIn) implies
```

```
        not (coilDriveIn.in1 == true & coilDriveIn.in2 == true) &
```

```
        not (coilDriveIn.in3 == true & coilDriveIn.in4 == true));
```

```
    guarantee MR_HLR_12 "1.12 motor: coil resistance is in 1..10 ohm window":
```

```
      coilResistance_mOhm >= 1000 & coilResistance_mOhm <= 10000;
```

```
    guarantee MR_HLR_13 "1.13 motor: rated current does not exceed 2 A":
```

```
      ratedCurrent_mA <= 2000;
```

```
  */
```

```
}
```

```
//
```

```
=====
```

```
=
```

```
// Processes
```

```
//
```

```
=====
```

```
=
```

```
part def ModeManager_sel4_i :> Process {
```

```
  port error_status: DataPort { in :>> type : ErrorStatusType; }
```

```
port mode: DataPort { out :>> type : ModeType; }
```

```
part ModeManager: ModeManager_i;
```

```
connection c1: PortConnection connect error_status to ModeManager.error_status;
```

```
connection c2: PortConnection connect ModeManager.mode to mode;
```

```
attribute :>> Domain = 2;
```

```
}
```

```
part def PowerMonitor_sel4_i :> Process {
```

```
port powerSampleIn: EventDataPort { in :>> type : PowerSample_i; }
```

```
port error_status: DataPort { out :>> type : ErrorStatusType; }
```

```
part PowerMonitor: PowerMonitor_i;
```

```
connection c1: PortConnection connect powerSampleIn to  
PowerMonitor.powerSampleIn;
```

```
connection c2: PortConnection connect PowerMonitor.error_status to error_status;
```

```
attribute :>> Domain = 3;
```

```
}
```

```
part def CameraDriver_sel4_i :> Process {
```

```
port cameraFrameOut: EventDataPort { out :>> type : CameraFrame_i; }
```

```

part CameraDriver:    CameraDriver_i;

part CameraDriverContract: CameraDriverContract_i;


connection c1: PortConnection connect CameraDriver.cameraFrameOut to
cameraFrameOut;

connection c2: PortConnection connect CameraDriverContract.cameraFrameOut to
cameraFrameOut;


attribute :>> Domain = 4;
}


part def MicroRosAgent_sel4_i :> Process {
  port stepCommandIn: EventDataPort { in :>> type : StepCommand_i; }
  port stepCommandOut: EventDataPort { out :>> type : StepCommand_i; }
  port cameraFrameIn: EventDataPort { in :>> type : CameraFrame_i; }
  port mode_in:    DataPort    { in :>> type : ModeType; }


  part MicroRosAgent: MicroRosAgent_i;


  connection c1: PortConnection connect stepCommandIn to
MicroRosAgent.stepCommandIn;

  connection c2: PortConnection connect MicroRosAgent.stepCommandOut to
stepCommandOut;

  connection c3: PortConnection connect cameraFrameIn to
MicroRosAgent.cameraFrameIn;

  connection c4: PortConnection connect mode_in    to MicroRosAgent.mode_in;

```



```
attribute :>> Domain = 5;  
}
```

```
part def StepperCoordinator_sel4_i :> Process {  
  port stepCommandIn: EventDataPort { in :>> type : StepCommand_i; }  
  port driveZ_out: EventDataPort { out :>> type : CoilDriveFrame_i; }  
  port driveX_out: EventDataPort { out :>> type : CoilDriveFrame_i; }  
  port driveY_out: EventDataPort { out :>> type : CoilDriveFrame_i; }
```

```
  part StepperCoordinator: StepperCoordinator_i;
```

```
  connection c1: PortConnection connect stepCommandIn to  
StepperCoordinator.stepCommandIn;
```

```
  connection c2: PortConnection connect StepperCoordinator.driveZ_out to driveZ_out;
```

```
  connection c3: PortConnection connect StepperCoordinator.driveX_out to driveX_out;
```

```
  connection c4: PortConnection connect StepperCoordinator.driveY_out to driveY_out;
```

```
  attribute :>> Domain = 6;  
}
```

```
// One Process definition reused for all three L298N boards (Z / X / Y)
```

```
part def L298N_Driver_sel4_i :> Process {  
  port coilDriveIn: EventDataPort { in :>> type : CoilDriveFrame_i; }  
  
  part L298N: L298N_Driver_i;
```

```
connection c1: PortConnection connect coilDriveIn to L298N.coilDriveIn;
```

```
attribute :>> Domain = 7;
```

```
}
```

```
// One Process definition reused for all three NEMA-17 motors
```

```
part def NEMA17_Stepper_sel4_i :> Process {
```

```
  port coilDriveIn: EventDataPort { in :>> type : CoilDriveFrame_i; }
```

```
  part Motor: NEMA17_Stepper_i;
```

```
  connection c1: PortConnection connect coilDriveIn to Motor.coilDriveIn;
```

```
  attribute :>> Domain = 8;
```

```
}
```

```
//
```

```
=====
=
```

```
// P r o c e s s o r s
```

```
//
```

```
=====
=
```

```
part def JetsonNano_Processor :> Processor {
```

```
  attribute :>> Frame_Period = 1000[ms];
```

```
  attribute :>> Clock_Period = 1[ms];
```

```
}
```

```
part def ESP32_Processor :> Processor {  
    attribute :>> Frame_Period = 100[ms];  
    attribute :>> Clock_Period = 1[ms];  
}
```

```
//
```

```
=====
```

```
=
```

```
// Top-Level System (mirrors the schematic)
```

```
//
```

```
=====
```

```
=
```

```
part def SeedImagingSystem_i :> System {
```

```
// ----- Processors -----
```

```
part jetson : JetsonNano_Processor;
```

```
part esp32 : ESP32_Processor;
```

```
// ----- Jetson-side processes -----
```

```
part ModeManager: ModeManager_seL4_i;
```

```
part PowerMonitor: PowerMonitor_seL4_i;
```

```
part CameraDriver: CameraDriver_seL4_i;    // USB camera (USB-A #2)
```

```
part MicroRosAgent: MicroRosAgent_seL4_i;    // micro-ROS host bridge
```

```

// ----- ESP32-side processes -----

part StepperCoordinator: StepperCoordinator_seL4_i;

part L298N_Z: L298N_Driver_seL4_i;          // Board #1 — IN1..IN4 on D2..D5
part L298N_X: L298N_Driver_seL4_i;          // Board #2 — IN1..IN4 on D6..D9
part L298N_Y: L298N_Driver_seL4_i;          // Board #3 — IN1..IN4 on D10..D13
part Motor_Z: NEMA17_Stepper_seL4_i;        // Turntable
part Motor_X: NEMA17_Stepper_seL4_i;        // Tilt
part Motor_Y: NEMA17_Stepper_seL4_i;        // Reserved Y


// ----- Wiring (mirrors the schematic / wiring guide) -----


// Camera (USB-A #2 on Jetson) -> micro-ROS Agent
connection cam1: PortConnection
    connect CameraDriver.cameraFrameOut to MicroRosAgent.cameraFrameIn;


// Mode + error status loop (Jetson)
connection ms1: PortConnection
    connect PowerMonitor.error_status to ModeManager.error_status;
connection ms2: PortConnection
    connect ModeManager.mode      to MicroRosAgent.mode_in;


// micro-ROS Agent (Jetson, USB-A #1) -> ESP32 Stepper Coordinator
connection sc1: PortConnection
    connect MicroRosAgent.stepCommandOut to StepperCoordinator.stepCommandIn;


// ESP32 GPIO -> L298N input pins ( D2..D5 / D6..D9 / D10..D13 )

```

```

connection gz: PortConnection
    connect StepperCoordinator.driveZ_out to L298N_Z.coilDriveIn;
connection gx: PortConnection
    connect StepperCoordinator.driveX_out to L298N_X.coilDriveIn;
connection gy: PortConnection
    connect StepperCoordinator.driveY_out to L298N_Y.coilDriveIn;

// L298N OUT1..OUT4 -> NEMA-17 coil pairs
connection mz: PortConnection
    connect L298N_Z.coilDriveIn to Motor_Z.coilDriveIn;
connection mx: PortConnection
    connect L298N_X.coilDriveIn to Motor_X.coilDriveIn;
connection my: PortConnection
    connect L298N_Y.coilDriveIn to Motor_Y.coilDriveIn;

// ----- Processor allocations -----
// Jetson hosts the high-level / vision / supervisory processes
allocation pb0: Deployment_Properties::Actual_Processor_Binding
    allocate ModeManager to jetson;
allocation pb1: Deployment_Properties::Actual_Processor_Binding
    allocate PowerMonitor to jetson;
allocation pb2: Deployment_Properties::Actual_Processor_Binding
    allocate CameraDriver to jetson;
allocation pb3: Deployment_Properties::Actual_Processor_Binding
    allocate MicroRosAgent to jetson;

```

```
// ESP32 hosts the real-time motor pipeline

allocation pb4: Deployment_Properties::Actual_Processor_Binding
    allocate StepperCoordinator to esp32;

allocation pb5: Deployment_Properties::Actual_Processor_Binding
    allocate L298N_Z to esp32;

allocation pb6: Deployment_Properties::Actual_Processor_Binding
    allocate L298N_X to esp32;

allocation pb7: Deployment_Properties::Actual_Processor_Binding
    allocate L298N_Y to esp32;

allocation pb8: Deployment_Properties::Actual_Processor_Binding
    allocate Motor_Z to esp32;

allocation pb9: Deployment_Properties::Actual_Processor_Binding
    allocate Motor_X to esp32;

allocation pb10: Deployment_Properties::Actual_Processor_Binding
    allocate Motor_Y to esp32;

}

}
```